WPF: основы

1. Desktop-разработка на .NET

Desktop-приложение — это программа, которая запускается локально на компьютере пользователя и имеет графический интерфейс: окна, кнопки, поля ввода, списки, меню. Пользователь взаимодействует с ней через мышь и клавиатуру, а не через командную строку. Именно такие приложения большинство людей запускает каждый день: текстовые редакторы, браузеры, IDE, мессенджеры, корпоративные системы учёта.

UI-фреймворки .NET

Написать desktop-приложение на C# можно разными способами. В .NET есть несколько UI-фреймворков, каждый со своей нишей.

WinForms — старейший фреймворк, интерфейс строится программно в C#-коде. Прост в освоении, но плохо масштабируется в крупных проектах и работает только на Windows.

WPF (Windows Presentation Foundation) — декларативный UI на XAML, который создавался на смену WinForms. До сих пор остаётся стандартом корпоративной разработки на .NET, работает только на Windows.

MAUI — современный кроссплатформенный фреймворк Microsoft, ориентированный прежде всего на мобильные устройства. Для desktop использует нативные контролы каждой платформы.

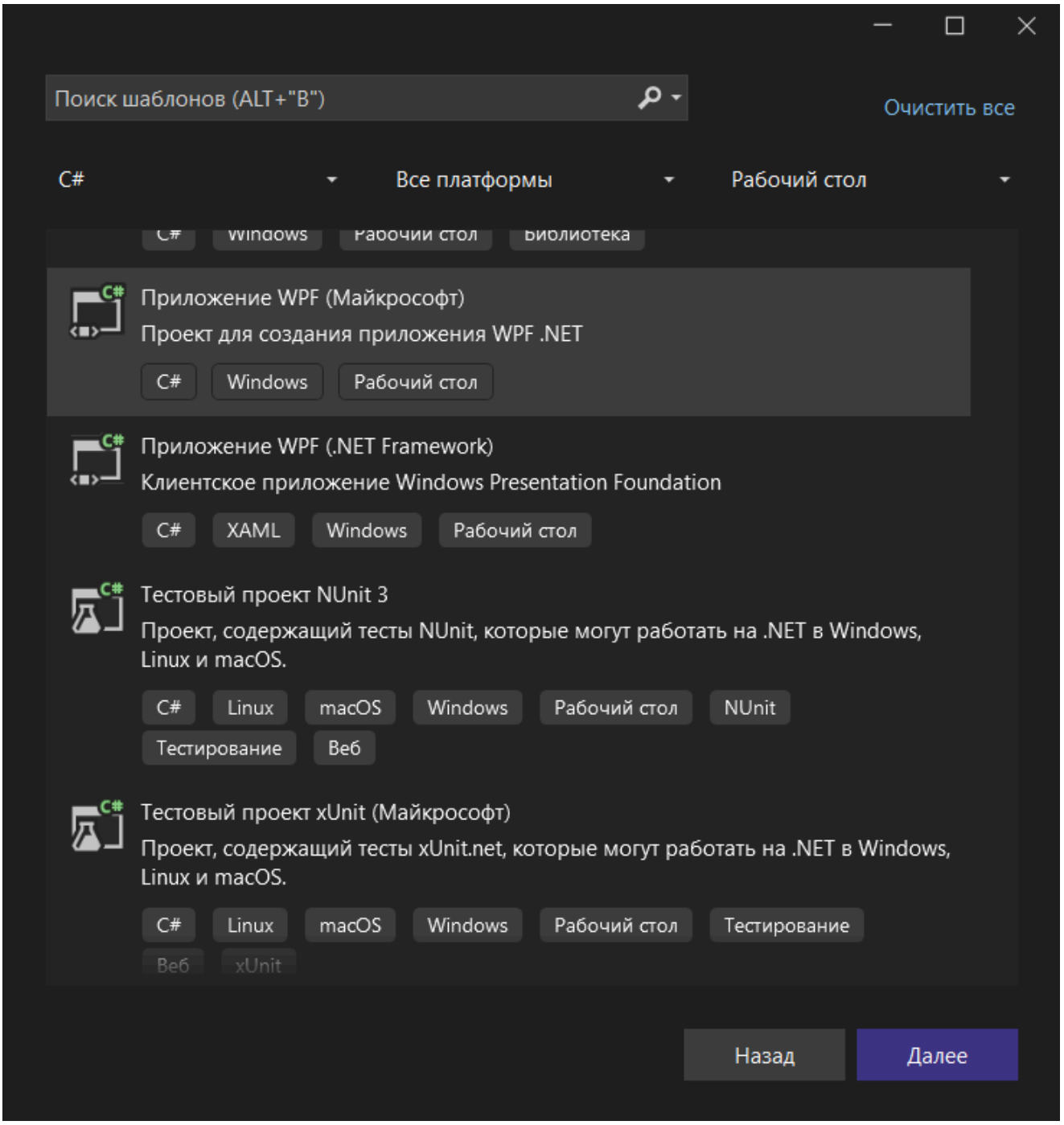
Avalonia — open-source фреймворк, вдохновлённый WPF, с поддержкой Windows, Linux и macOS. Рисует интерфейс самостоятельно, не полагаясь на нативные контролы ОС.

В этой и следующей лекции мы сосредоточимся на WPF и будем строить систему учёта книг — проект `LibraryApp`.

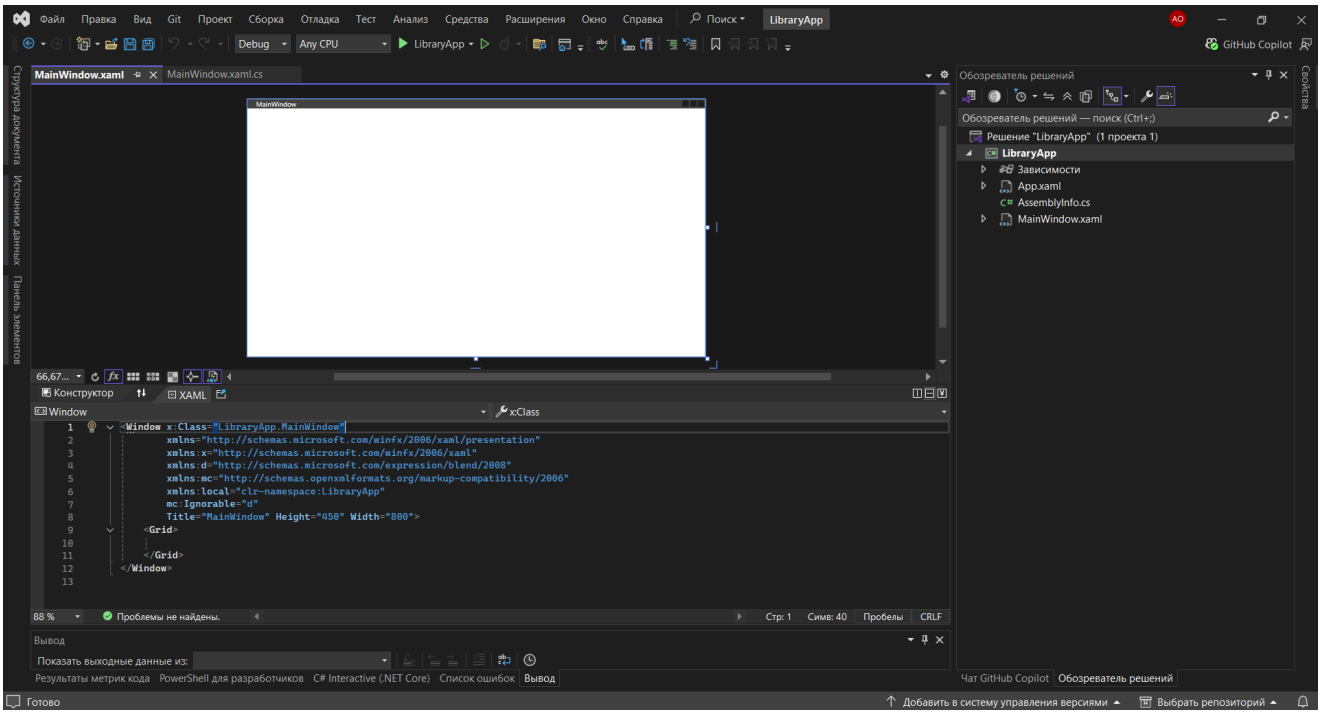
2. Создание проекта и структура приложения

Создание проекта

Для создания WPF-проекта откройте Visual Studio и выберите шаблон **Приложение WPF (Майкрософт)**.



Назовите проект `LibraryApp` и создайте его. В центре экрана откроется XAML-редактор с `MainWindow.xaml` — экран разделён на две части: разметка внизу, визуальный превью вверху.

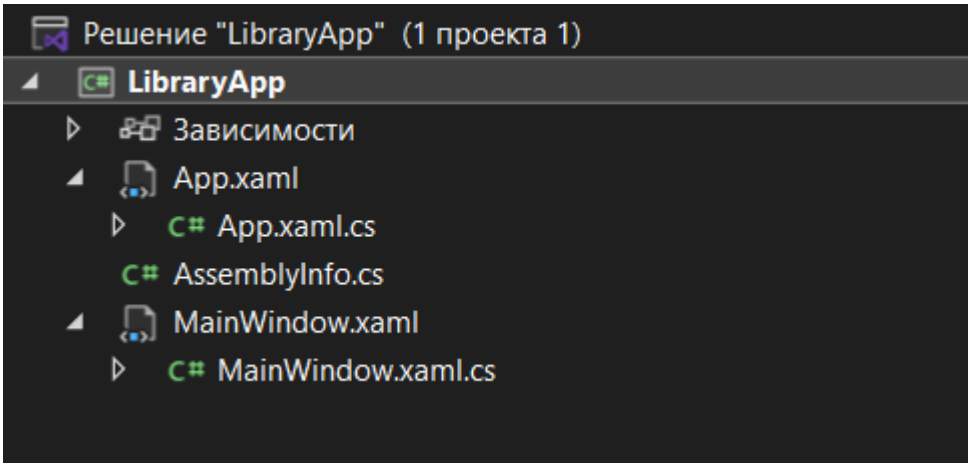


Запустите приложение, чтобы убедиться что всё работает. Откроется пустое окно с заголовком «MainWindow».

Структура проекта

```
LibraryApp/
├── App.xaml              — глобальная конфигурация приложения
├── App.xaml.cs           — жизненный цикл приложения
├── MainWindow.xaml       — разметка главного окна
├── MainWindow.xaml.cs    — code-behind для MainWindow
└── LibraryApp.csproj     — файл проекта
```

Каждый `.xaml`-файл имеет парный `.cs`-файл — они образуют единый класс через механизм `partial`. Чтобы увидеть вложенный `.cs`-файл, раскройте стрелку рядом с `.xaml` в обозревателе решений.



App.xaml

`App.xaml` — корневой элемент приложения. Здесь указывается какое окно открывать при старте и объявляются глобальные ресурсы, доступные во всём приложении.

```
<Application x:Class="LibraryApp.App"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             StartupUri="MainWindow.xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```

`StartupUri="MainWindow.xaml"` указывает, какое окно открыть при запуске. В `Application.Resources` позже будем размещать глобальные стили, доступные из любого места приложения.

App.xaml.cs

```
using System.Windows;

namespace LibraryApp
{
    public partial class App : Application
    {
    }
}
```

Этот класс становится нужен когда требуется перехватить события жизненного цикла приложения — запуск, завершение, необработанные исключения. Например, вместо `StartupUri` можно создать главное окно вручную в `OnStartup`:

```
public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        var window = new MainWindow();
        window.Show();
    }
}
```

MainWindow.xaml — разметка окна

```
<Window x:Class="LibraryApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="LibraryApp"
        Width="800"
        Height="450">
    <Grid>

    </Grid>
</Window>
```

`Window` — корневой элемент для окна приложения. Внутри него может быть только один дочерний элемент — по умолчанию это `Grid`. Атрибут `x:Class="LibraryApp.MainWindow"` связывает этот XAML-файл с C#-классом `MainWindow` — подробнее об этом ниже.

MainWindow.xaml.cs — code-behind окна

```
using System.Windows;

namespace LibraryApp
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }
    }
}
```

Этот файл — **code-behind**: C#-класс, связанный с XAML-разметкой. Слово `partial` означает, что класс `MainWindow` разделён на две части. Первая — этот файл, который мы видим и редактируем. Вторая — генерируется WPF из `MainWindow.xaml` при сборке: она содержит реализацию `InitializeComponent()` и поля для всех элементов с атрибутом `x:Name`. Компилятор объединяет обе части в один класс.

`InitializeComponent()` — не формальность. Без него элементы из XAML не создадутся: окно откроется пустым, а обращения к именованным элементам выбросят `NullReferenceException`.

Поток инициализации

Вот что происходит при запуске приложения:

```
Запуск приложения
    │
    ▼
App.xaml обрабатывается
    │  Читается StartupUri — определяется стартовое окно
    │  Загружаются глобальные ресурсы из Application.Resources
    │
    ▼
App.xaml.cs — OnStartup()
    │  Если переопределён — выполняется код инициализации
    │  Создаётся главное окно (явно или через StartupUri)
    │
    ▼
MainWindow.xaml.cs — конструктор
    │  InitializeComponent() — загружает MainWindow.xaml
    │  Создаются все элементы интерфейса
    │
    ▼
Окно отображается на экране
    │
    ▼
Message loop — приложение реагирует на действия пользователя
```

Понимание этого потока помогает при отладке: если что-то не работает при старте, нужно понимать на каком именно шаге происходит сбой.

Организация папок проекта

Базовый шаблон помещает все файлы в корень проекта. Для небольшого проекта это приемлемо, но когда файлов станет больше — ориентироваться в плоском списке неудобно. Организуем файлы по назначению сразу.

Через обозреватель решений создадим папки `Views` и `Resources`. Перетащим `MainWindow.xaml` в папку `Views` — Visual Studio обновит пространство имён

автоматически. После переноса обновим `StartupUri` в `App.xaml` вручную:

```
StartupUri="Views/MainWindow.xaml"
```

В дальнейшем проект будет выглядеть так:

```
LibraryApp/
├── Views/
│   ├── MainWindow.xaml
│   └── MainWindow.xaml.cs
├── Resources/
│   └── ...   — стили
├── App.xaml
├── App.xaml.cs
└── LibraryApp.csproj
```

Папку `Resources` и её содержимое мы заполним в разделе 6, когда будем разбирать стили.

3. XAML — декларативное описание интерфейса

Пользовательский интерфейс в WPF описывается на языке XAML. Прежде чем переходить к конкретным элементам управления, важно понять саму идею декларативного подхода и разобраться с синтаксисом, на котором строится весь UI.

Декларативный vs императивный подход

Существует два способа создать кнопку в приложении.

Императивный — описываем *как* создать объект, шаг за шагом в коде:

```
var button = new Button();
button.Content = "Сохранить";
button.Width = 120;
button.HorizontalAlignment = HorizontalAlignment.Center;
button.Margin = new Thickness(0, 10, 0, 0);
```

Декларативный — описываем *что* должно быть на экране, в разметке:

```
<Button Content="Сохранить"
        Width="120"
        HorizontalAlignment="Center"
        Margin="0,10,0,0" />
```

Результат одинаковый, но читаемость разная. В декларативном подходе структура интерфейса видна сразу, в отличие от императивного кода, где нужно мысленно выполнять последовательность операций, чтобы представить результат.

Синтаксис: элементы и атрибуты

Каждый элемент интерфейса в WPF представлен XML-тегом. Свойства задаются атрибутами:

```
<TextBlock Text="Привет, мир!"
           FontSize="18"
           Foreground="DarkBlue"
           FontWeight="Bold" />
```

Здесь `TextBlock` — элемент управления для отображения текста, а `Text`, `FontSize`, `Foreground`, `FontWeight` — его свойства.

Самозакрывающийся тег (`/>`) используется когда элемент не содержит дочерних элементов. Если дочерние элементы есть — используется пара открывающего и закрывающего тегов:

```
<StackPanel>
    <TextBlock Text="Название книги" />
    <TextBlock Text="Автор" />
    <Button Content="Редактировать" />
</StackPanel>
```

Пространства имён

Каждый XAML-файл начинается с объявления пространств имён в корневом теге. Без них парсер не знает, к каким классам относятся теги разметки:

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="clr-namespace:LibraryApp.Views"
        x:Class="LibraryApp.Views.MainWindow">
</Window>
```

`xmlns` — основное пространство имён WPF. Все стандартные контролы: `Button`, `TextBox`, `Grid`, `StackPanel`.

`xmlns:x` — пространство имён для служебных конструкций самого языка XAML. Всё с префиксом `x:` — это директивы языка, а не WPF: `x:Class`, `x:Name`, `x:Key`, `x:Type`.

`xmlns:local` — подключение собственных классов приложения. После этого объявления можно использовать классы из пространства имён `LibraryApp.Views` с префиксом `local:`.

x:Class — связь XAML с C#

Атрибут `x:Class` связывает XAML-файл с C#-классом:

```
<Window x:Class="LibraryApp.Views.MainWindow">
```

```
// MainWindow.xaml.cs
namespace LibraryApp.Views
{
    public partial class MainWindow : Window { ... }
}
```

Значение `x:Class` должно точно совпадать с пространством имён и именем класса в `.cs`-файле. Если они расходятся — проект не скомпилируется.

x:Name — имя элемента

Любому элементу можно дать имя, чтобы обратиться к нему из code-behind:

```
<TextBox x:Name="TitleBox" />
```

```
// В code-behind
TitleBox.Focus();
```

При сборке WPF генерирует поле с этим именем в `partial`-классе. Имя должно быть уникальным в пределах одного файла.

`x:Name` используется для UI-специфичных задач: установки фокуса, запуска анимации. Для передачи данных между интерфейсом и логикой используются привязки данных — о них в следующей лекции.

Комментарии

Комментарии используют стандартный XML-синтаксис:

```
<!-- Это однострочный комментарий -->

<!--
    Это многострочный комментарий.
    Удобно для временного отключения фрагмента разметки.
-->
<Button Content="Активная кнопка" />
<!-- <Button Content="Временно отключённая кнопка" /> -->
```

Вложенные свойства

Простые значения задаются атрибутами. Но некоторые свойства требуют более сложных значений — объектов или коллекций. Для этого используется синтаксис вложенного элемента свойства:

```
<!-- Простое свойство — атрибутом -->
<Button Content="Сохранить" />

<!-- Сложное свойство — вложенным элементом -->
<Button>
    <Button.Content>
        <StackPanel Orientation="Horizontal">
            <TextBlock Text="💾" VerticalAlignment="Center" />
            <TextBlock Text="Сохранить"
                       VerticalAlignment="Center"
                       Margin="6,0,0,0" />
        </StackPanel>
    </Button.Content>
</Button>
```

Синтаксис `ТипЭлемента.НазваниеСвойства` — запись вложенного свойства. Используется когда значение свойства — не строка, а объект или набор элементов.

Присоединённые свойства

WPF поддерживает **присоединённые свойства** — свойства, которые определены в одном классе, но применяются к другому. Самый распространённый пример — свойства `Grid`, которые указывают в какой строке и столбце находится элемент:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
    </Grid.RowDefinitions>

    <!-- Grid.Row определён в классе Grid, но применяется к TextBlock -->
    <TextBlock Text="Заголовок" Grid.Row="0" />
    <TextBox Grid.Row="1" />
</Grid>
```

`Grid.Row="0"` — присоединённое свойство. Синтаксис: `ИмяКласса.ИмяСвойства`. Другие примеры: `DockPanel.Dock`, `Canvas.Left`, `Canvas.Top`.

4. Панели компоновки

Панели компоновки — это контейнеры, которые управляют расположением дочерних элементов. В WPF нет абсолютного позиционирования как основного подхода — вместо этого элементы размещаются относительно друг друга и родительского контейнера. Это делает интерфейс адаптивным: он корректно выглядит при изменении размера окна.

Margin и Padding

Прежде чем переходить к панелям, познакомимся с двумя свойствами для управления отступами — они используются у всех элементов.

`Margin` — внешний отступ: расстояние от элемента до соседних элементов и краёв контейнера.

`Padding` — внутренний отступ: расстояние от краёв элемента до его содержимого.

```
<!-- Margin: все стороны по 10 -->
<Button Content="Равные отступы" Margin="10" />

<!-- Margin: лево/право 16, верх/низ 8 -->
<Button Content="Горизонтальные больше" Margin="16,8" />

<!-- Margin: лево 0, верх 8, право 0, низ 8 — порядок: лево, верх, право,
низ -->
<Button Content="Только вертикальные" Margin="0,8,0,8" />

<!-- Padding внутри Border -->
<Border Padding="16,12"
        Background="LightGray"
        CornerRadius="4">
    <TextBlock Text="Текст с отступом внутри рамки" />
</Border>
```

Значения задаются в одном из трёх форматов:

- `"10"` — одно значение для всех сторон
- `"16,8"` — горизонтальные, вертикальные
- `"0,8,0,8"` — лево, верх, право, низ

HorizontalAlignment и VerticalAlignment

Управляют выравниванием элемента внутри отведённого ему пространства:

```
<StackPanel Width="300">
    <Button Content="Left"    HorizontalAlignment="Left"    />
    <Button Content="Center"  HorizontalAlignment="Center"  />
    <Button Content="Right"   HorizontalAlignment="Right"   />
    <!-- Stretch растягивает элемент на всю доступную ширину -->
    <Button Content="Stretch" HorizontalAlignment="Stretch" />
</StackPanel>
```

`VerticalAlignment` работает аналогично: `Top`, `Center`, `Bottom`, `Stretch`. Применяется когда элемент находится в контейнере с доступным пространством по вертикали — например в `Grid` со строкой фиксированной высоты.

StackPanel

`StackPanel` укладывает дочерние элементы последовательно — один за другим, вертикально или горизонтально. Это самая простая и часто используемая панель.

Вертикальная укладка (по умолчанию):

```
<StackPanel>
    <TextBlock Text="Имя" />
    <TextBox Margin="0,4,0,0" />
    <TextBlock Text="Email" Margin="0,12,0,0" />
    <TextBox Margin="0,4,0,0" />
    <Button Content="Сохранить"
            Margin="0,16,0,0" />
</StackPanel>
```

Горизонтальная укладка:

```
<StackPanel Orientation="Horizontal">
    <Button Content="Сохранить" />
    <Button Content="Отмена"    Margin="8,0,0,0" />
    <Button Content="Удалить"   Margin="8,0,0,0" />
</StackPanel>
```

Grid

`Grid` — самая мощная и универсальная панель. Она разбивает пространство на строки и столбцы, и каждый элемент помещается в конкретную ячейку.

Объявление строк и столбцов:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="120" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
</Grid>
```

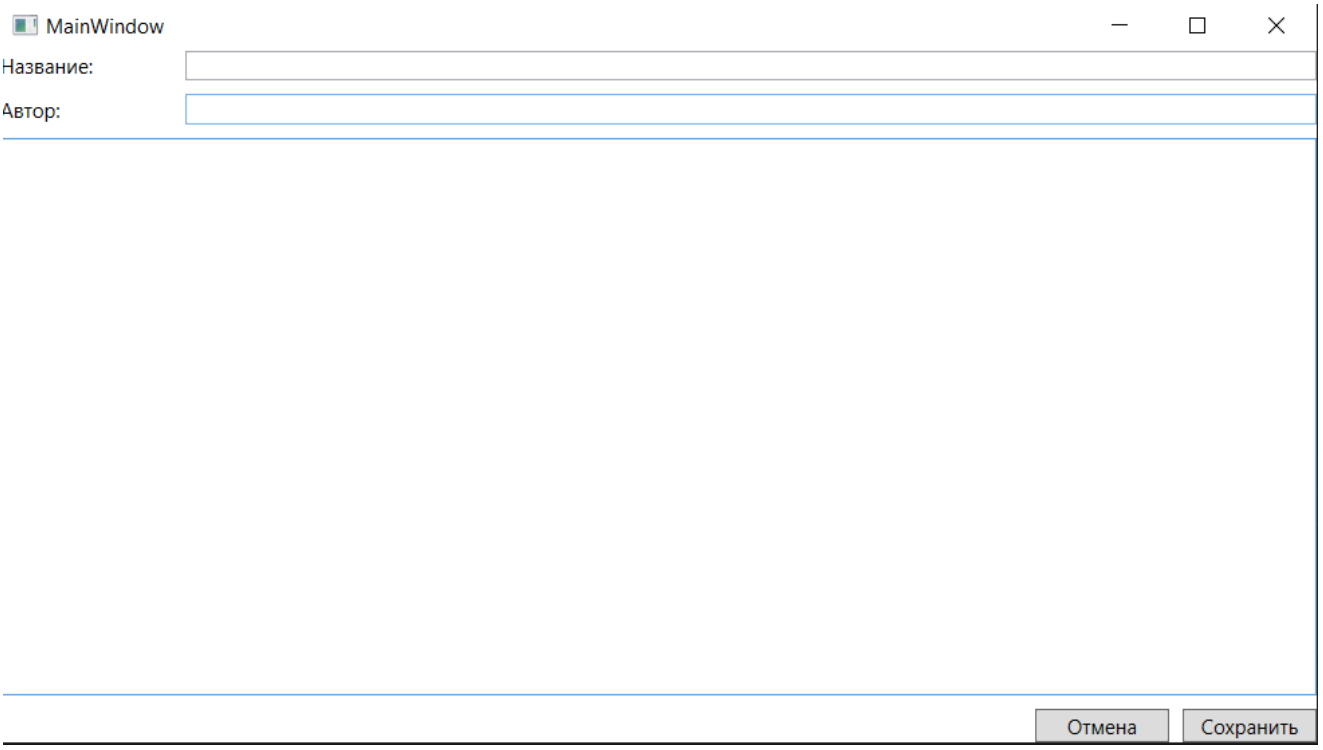
Размеры задаются через специальные значения:

- `Auto` — размер подстраивается под содержимое
- `*` — занять всё оставшееся пространство
- `2*` — занять долю оставшегося пространства (в два раза больше чем `*`)
- `200` — фиксированный размер в пикселях

Размещение элементов в ячейках осуществляется через присоединённые свойства `Grid.Row` и `Grid.Column`:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="*" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="110" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <TextBlock Text="Название:"
               Grid.Row="0" Grid.Column="0"
               VerticalAlignment="Center" />
    <TextBox   Grid.Row="0" Grid.Column="1" />

    <TextBlock Text="Автор:"
```

```
                Grid.Row="1" Grid.Column="0"
                VerticalAlignment="Center"
                Margin="0,8,0,0" />
    <TextBox    Grid.Row="1" Grid.Column="1"
                Margin="0,8,0,0" />

    <!-- TextBox занимает оба столбца через Grid.ColumnSpan -->
    <TextBox Grid.Row="2" Grid.Column="0"
             Grid.ColumnSpan="2"
             AcceptsReturn="True"
             TextWrapping="Wrap"
             Margin="0,8,0,0" />

    <!-- Кнопки в правой части последней строки -->
    <StackPanel Grid.Row="3" Grid.Column="1"
                Orientation="Horizontal"
                HorizontalAlignment="Right"
                Margin="0,8,0,0">
        <Button Content="Отмена" Width="80" />
        <Button Content="Сохранить" Width="80"
                Margin="8,0,0,0" />
    </StackPanel>

</Grid>
```



`Grid.ColumnSpan` и `Grid.RowSpan` позволяют элементу занимать несколько столбцов или строк:

```
<!-- Заголовок занимает оба столбца -->
<TextBlock Text="Добавление книги"
```

```
            Grid.Row="0" Grid.Column="0"
            Grid.ColumnSpan="2"
            FontSize="18"
            FontWeight="SemiBold"
            Margin="0,0,0,16" />
```

Располагать несколько элементов в одной ячейке — допустимо, они накладываются друг на друга. Порядок в разметке определяет порядок отрисовки (последний — поверх):

```
<Grid>
    <Image Source="/Assets/background.jpg"
           Stretch="UniformToFill" />
    <TextBlock Text="Текст поверх изображения"
               HorizontalAlignment="Center"
               VerticalAlignment="Center"
               Foreground="White"
               FontSize="24" />
</Grid>
```

DockPanel

`DockPanel` прикрепляет элементы к сторонам контейнера. Это типичная панель для построения основного layout приложения: меню сверху, статусная строка снизу, панель навигации слева, основной контент занимает оставшееся место.

```
<DockPanel>

    <!-- Меню прикреплено к верхнему краю -->
    <Menu DockPanel.Dock="Top">
        <MenuItem Header="Файл">
            <MenuItem Header="Открыть" />
            <MenuItem Header="Сохранить" />
            <Separator />
            <MenuItem Header="Выход" />
        </MenuItem>
        <MenuItem Header="Правка">
            <MenuItem Header="Найти" />
        </MenuItem>
    </Menu>

    <!-- Статусная строка прикреплена к нижнему краю -->
    <Border DockPanel.Dock="Bottom"
            BorderBrush="#E0E0E0"
            BorderThickness="0,1,0,0"
            Padding="8,4">
        <TextBlock Text="Готово" FontSize="12" />
```

```
    </Border>

    <!-- Панель навигации прикреплена к левому краю -->
    <Border DockPanel.Dock="Left"
            Width="200"
            BorderBrush="#E0E0E0"
            BorderThickness="0,0,1,0"
            Padding="8">
        <StackPanel>
            <TextBlock Text="Навигация"
                       FontWeight="SemiBold"
                       Margin="0,0,0,8" />
            <Button Content="Книги"
                    HorizontalAlignment="Stretch" />
            <Button Content="Авторы"
                    HorizontalAlignment="Stretch"
                    Margin="0,4,0,0" />
        </StackPanel>
    </Border>

    <!-- Последний элемент без Dock занимает весь оставшийся центр -->
    <TextBlock Text="Основной контент"
               HorizontalAlignment="Center"
               VerticalAlignment="Center" />

</DockPanel>
```

Порядок элементов в `DockPanel` важен: каждый следующий прикреплённый элемент получает пространство из того, что осталось после предыдущих. Последний элемент без `DockPanel.Dock` всегда занимает весь оставшийся центр.

WrapPanel

`WrapPanel` укладывает элементы в строку и переносит на новую строку, когда место заканчивается — как слова в тексте. Полезен для тегов, кнопок-фильтров, галерей:

```
<WrapPanel Orientation="Horizontal">
    <Button Content="Все"        Margin="4" Padding="8,4" />
    <Button Content="Классика"   Margin="4" Padding="8,4" />
    <Button Content="Фантастика" Margin="4" Padding="8,4" />
    <Button Content="Детектив"   Margin="4" Padding="8,4" />
    <Button Content="История"    Margin="4" Padding="8,4" />
    <Button Content="Биография"  Margin="4" Padding="8,4" />
</WrapPanel>
```

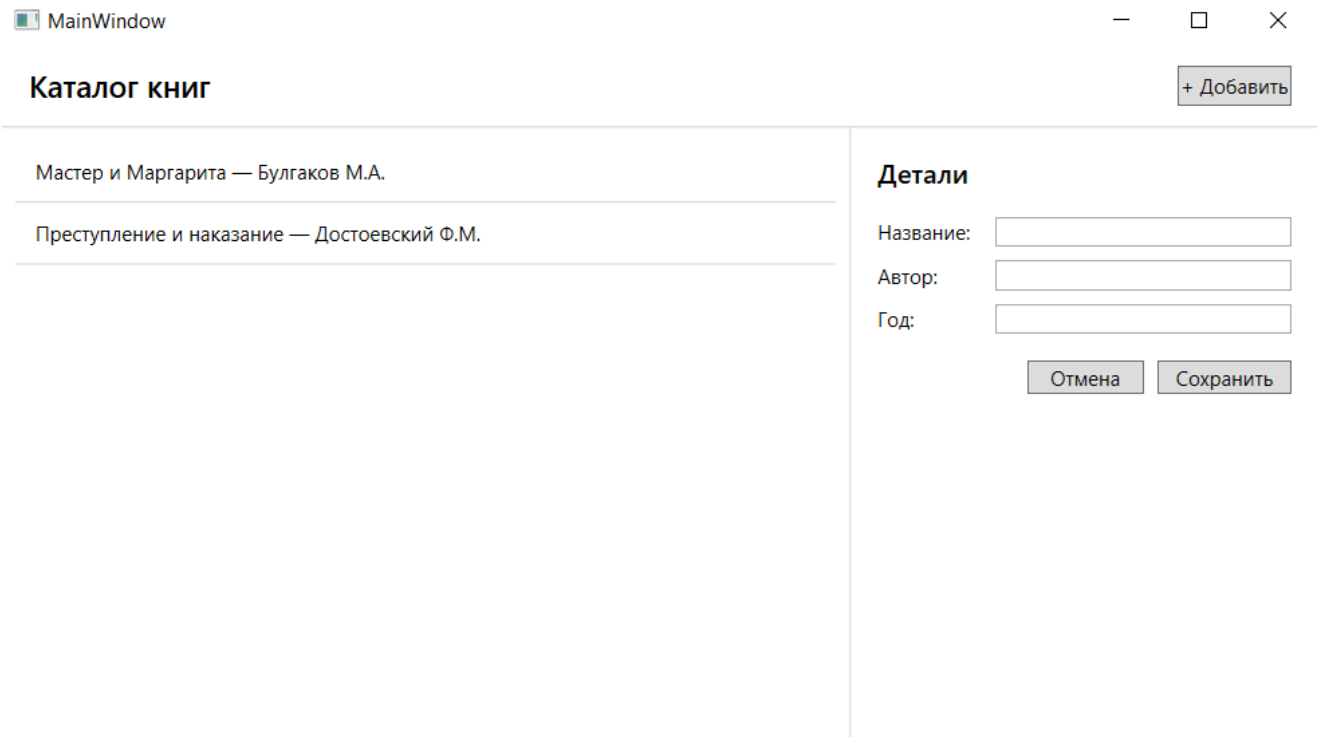
При уменьшении окна кнопки автоматически переносятся на следующую строку.

Вложенные панели

Реальные интерфейсы строятся из комбинации нескольких панелей. Каждая отвечает за свою область:

```
<!-- DockPanel задаёт основной layout: шапка, контент, список -->
<DockPanel>

    <!-- Шапка с заголовком и кнопкой добавления -->
    <Border DockPanel.Dock="Top"
            Padding="16,12"
            BorderBrush="#E0E0E0"
            BorderThickness="0,0,0,1">
        <DockPanel>
            <TextBlock DockPanel.Dock="Left"
                       Text="Каталог книг"
                       FontSize="18"
                       FontWeight="SemiBold"
                       VerticalAlignment="Center" />
            <Button DockPanel.Dock="Right"
                    Content="+ Добавить"
                    HorizontalAlignment="Right" />
        </DockPanel>
    </Border>

    <!-- Панель деталей прикреплена справа -->
    <Border DockPanel.Dock="Right"
            Width="280"
            Padding="16"
            BorderBrush="#E0E0E0"
            BorderThickness="1,0,0,0">
        <StackPanel>
            <TextBlock Text="Детали"
                       FontSize="16"
                       FontWeight="SemiBold"
                       Margin="0,0,0,16" />

            <!-- Grid выравнивает метки и поля -->
            <Grid>
                <Grid.RowDefinitions>
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                    <RowDefinition Height="Auto" />
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="70" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>

                <TextBlock Text="Название:"
                           Grid.Row="0" Grid.Column="0"
```

```
                            VerticalAlignment="Center" />
                <TextBox    Grid.Row="0" Grid.Column="1" />

                <TextBlock Text="Автор:"
                           Grid.Row="1" Grid.Column="0"
                           VerticalAlignment="Center"
                           Margin="0,8,0,0" />
                <TextBox    Grid.Row="1" Grid.Column="1"
                           Margin="0,8,0,0" />

                <TextBlock Text="Год:"
                           Grid.Row="2" Grid.Column="0"
                           VerticalAlignment="Center"
                           Margin="0,8,0,0" />
                <TextBox    Grid.Row="2" Grid.Column="1"
                           Margin="0,8,0,0" />
            </Grid>

            <!-- Кнопки выровнены по правому краю -->
            <StackPanel Orientation="Horizontal"
                        HorizontalAlignment="Right"
                        Margin="0,16,0,0">
                <Button Content="Отмена"   Width="70" />
                <Button Content="Сохранить" Width="80"
                        Margin="8,0,0,0" />
            </StackPanel>
        </StackPanel>
    </Border>

    <!-- Список книг занимает оставшееся пространство -->
    <ScrollViewer VerticalScrollBarVisibility="Auto">
        <StackPanel Margin="8">
            <Border Padding="12,10"
                    BorderBrush="#E0E0E0"
                    BorderThickness="0,0,0,1">
                <TextBlock Text="Мастер и Маргарита — Булгаков М.А." />
            </Border>
            <Border Padding="12,10"
                    BorderBrush="#E0E0E0"
                    BorderThickness="0,0,0,1">
                <TextBlock Text="Преступление и наказание — Достоевский
Ф.М." />
            </Border>
        </StackPanel>
    </ScrollViewer>

</DockPanel>
```



В этом примере три панели работают вместе: `DockPanel` задаёт общий layout страницы, внутри неё `Grid` управляет формой редактирования, а `StackPanel` выстраивает кнопки по горизонтали.

Как выбрать панель

Задача	Панель
Выстроить элементы в столбец или строку	`StackPanel`
Разбить пространство на зоны (шапка, контент, подвал)	`DockPanel`
Форма с метками и полями в сетке	`Grid`
Наложить элементы друг на друга	`Grid` (несколько элементов в одной ячейке)
Теги, плитки с переносом	`WrapPanel`

Правило выбора простое: начинайте с самой простой панели, которая решает задачу. Если `StackPanel` справляется — не используйте `Grid`. Если элементы нужно выровнять по сетке — `Grid` предпочтительнее вложенных `StackPanel`.

5. Базовые элементы управления

Элементы управления — строительные блоки пользовательского интерфейса. Каждый отвечает за конкретную задачу: отобразить текст, принять ввод, показать список, выполнить действие. В этом разделе разберём основные контролы WPF и покажем, как они работают вместе в реальном интерфейсе.

TextBlock — отображение текста

`TextBlock` — основной элемент для отображения текста, доступного только для чтения. Используется везде: заголовки, подписи, метки полей, статусные сообщения.

```
<!-- Простой текст -->
<TextBlock Text="Привет, мир!" />

<!-- Настройка шрифта -->
<TextBlock Text="Заголовок страницы"
           FontSize="24"
           FontWeight="SemiBold"
           Foreground="#1A1A1A" />

<!-- Перенос длинного текста -->
<TextBlock Text="Это очень длинный текст, который не помещается в одну
строку"
           TextWrapping="Wrap"
           MaxWidth="300" />

<!-- Усечение с многоточием -->
<TextBlock Text="Очень длинное название книги которое не помещается"
           TextTrimming="CharacterEllipsis"
           Width="200" />
```

`TextWrapping="Wrap"` включает автоматический перенос текста. `TextTrimming="CharacterEllipsis"` усекает текст и добавляет `…` в конце — удобно для списков с фиксированной шириной.

`TextBlock` также поддерживает **форматированный текст** через вложенные элементы `Run`, `Bold`, `Italic`:

```
<TextBlock>
    <Run Text="Обычный текст, " />
    <Bold>жирный текст, </Bold>
    <Italic>курсив, </Italic>
    <Run Text="снова обычный" Foreground="Gray" FontSize="12" />
</TextBlock>
```

TextBox — ввод текста

`TextBox` — поле для ввода и редактирования текста пользователем.

```
<!-- Простое поле ввода -->
<TextBox />

<!-- С ограничением длины -->
<TextBox MaxLength="100" />

<!-- Многострочный ввод -->
<TextBox AcceptsReturn="True"
         TextWrapping="Wrap"
         Height="120"
         VerticalScrollBarVisibility="Auto" />

<!-- Только для чтения -->
<TextBox Text="Нельзя изменить"
         IsReadOnly="True" />
```

PasswordBox — ввод пароля

Для ввода пароля в WPF используется отдельный элемент `PasswordBox`, а не `TextBox` с маскировкой символов:

```
<PasswordBox x:Name="PasswordInput"
             MaxLength="50" />
```

```
// Пароль читается через свойство Password, а не Text
string password = PasswordInput.Password;
```

Button — кнопка

`Button` инициирует действие при нажатии.

```
<!-- Простая кнопка -->
<Button Content="Сохранить"
        Width="100" />

<!-- Растянуть на всю ширину -->
<Button Content="Войти"
        HorizontalAlignment="Stretch" />

<!-- Отключённая кнопка -->
<Button Content="Недоступно"
        IsEnabled="False" />
```

```
<!-- Кнопка со сложным содержимым -->
<Button Width="130" Padding="8,6">
    <StackPanel Orientation="Horizontal">
        <TextBlock Text="💾" FontSize="14"
                   VerticalAlignment="Center" />
        <TextBlock Text="Сохранить"
                   VerticalAlignment="Center"
                   Margin="8,0,0,0" />
    </StackPanel>
</Button>
```

`Content` кнопки может быть не только строкой, но и любым элементом — можно вложить иконку, изображение или произвольную разметку.

WPF различает два специальных типа кнопки через свойства `IsDefault` и `IsCancel`:

```
<!-- IsDefault — срабатывает при нажатии Enter -->
<Button Content="OK"     IsDefault="True" />

<!-- IsCancel — срабатывает при нажатии Escape -->
<Button Content="Отмена" IsCancel="True" />
```

Эти свойства удобны для диалоговых окон: пользователь может подтвердить действие через Enter или отменить через Escape без клика мышью.

CheckBox — флажок

`CheckBox` — независимый флажок, может быть установлен или снят:

```
<StackPanel>
    <CheckBox Content="Запомнить меня" />
    <CheckBox Content="Получать уведомления"
              IsChecked="True"
              Margin="0,8,0,0" />
    <CheckBox Content="Недоступный флажок"
              IsEnabled="False"
              Margin="0,8,0,0" />
</StackPanel>
```

WPF поддерживает три состояния `CheckBox` через `IsThreeState="True"`: установлен, снят и неопределён (`null`). Неопределённое состояние полезно для «родительских» флажков, когда часть дочерних элементов выбрана, а часть нет:

```
<CheckBox Content="Выбрать все"
          IsThreeState="True"
```

```
            IsChecked="{x:Null}" />
```

RadioButton — переключатель

`RadioButton` — зависимый выбор, только один из группы может быть выбран. Группировка через `GroupName`:

```
<StackPanel>
    <TextBlock Text="Сортировка:"
               FontWeight="SemiBold"
               Margin="0,0,0,8" />
    <RadioButton Content="По названию"
                 GroupName="Sort"
                 IsChecked="True" />
    <RadioButton Content="По автору"
                 GroupName="Sort"
                 Margin="0,4,0,0" />
    <RadioButton Content="По году"
                 GroupName="Sort"
                 Margin="0,4,0,0" />
</StackPanel>
```

Без `GroupName` все `RadioButton` в одном контейнере автоматически образуют группу. `GroupName` нужен когда в одном контейнере находится несколько независимых групп переключателей.

ComboBox — выпадающий список

`ComboBox` позволяет выбрать одно значение из фиксированного набора вариантов:

```
<ComboBox Width="200">
    <ComboBoxItem Content="Все жанры" IsSelected="True" />
    <ComboBoxItem Content="Классика" />
    <ComboBoxItem Content="Фантастика" />
    <ComboBoxItem Content="Детектив" />
    <ComboBoxItem Content="История" />
</ComboBox>
```

`ComboBox` можно сделать редактируемым — пользователь сможет не только выбирать из списка, но и вводить своё значение:

```
<ComboBox IsEditable="True"
          Width="200" />
```

ProgressBar — индикатор прогресса

```
<!-- Определённый прогресс — 70% -->
<ProgressBar Minimum="0"
             Maximum="100"
             Value="70"
             Height="8" />

<!-- Неопределённый прогресс — анимированная полоска -->
<ProgressBar IsIndeterminate="True"
             Height="8" />
```

`IsIndeterminate="True"` показывает бегущую анимацию, когда точный прогресс неизвестен — например при загрузке данных из базы.

ScrollViewer — прокрутка

`ScrollViewer` оборачивает содержимое и добавляет полосы прокрутки, когда содержимое не помещается в отведённую область:

```
<ScrollViewer Height="200"
              VerticalScrollBarVisibility="Auto"
              HorizontalScrollBarVisibility="Disabled">
    <StackPanel>
        <!-- много элементов -->
    </StackPanel>
</ScrollViewer>
```

`VerticalScrollBarVisibility` и `HorizontalScrollBarVisibility` принимают значения:

- `Auto` — показывать полосу только когда нужна
- `Visible` — всегда показывать
- `Hidden` — скрыть полосу, но прокрутку оставить
- `Disabled` — отключить прокрутку полностью

Полный пример: форма редактирования книги

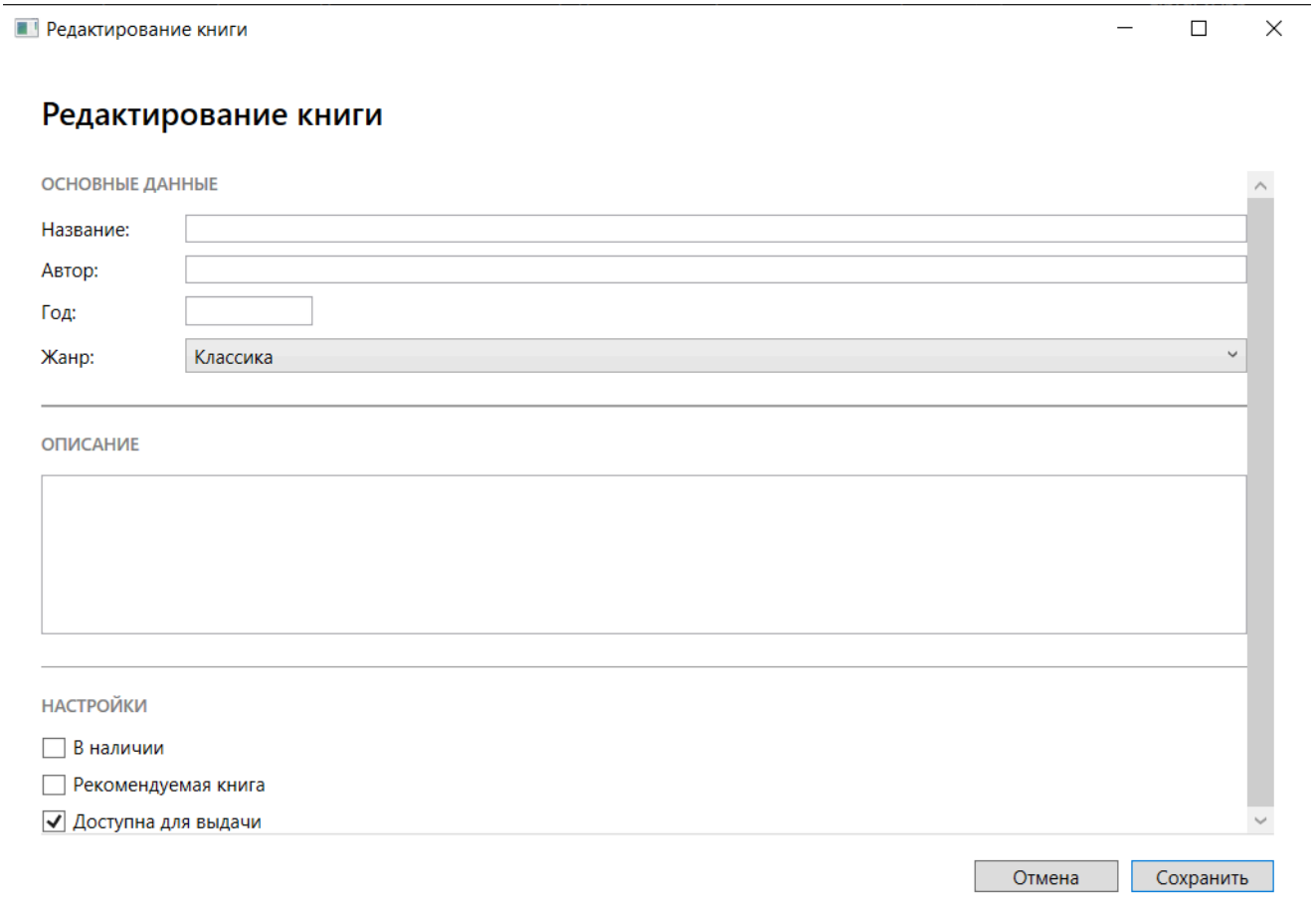
Соберём все изученные элементы в одну форму. Это отдельное окно — `BookEditWindow`. Создайте его в папке `Views`: правой кнопкой на папку → **Add** → **New Item** → **Window (WPF)**.

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="LibraryApp.Views.BookEditWindow"
        Title="Редактирование книги"
        Width="480"
        Height="580"
        WindowStartupLocation="CenterScreen"
        ResizeMode="NoResize">

    <DockPanel Margin="24">

        <!-- Заголовок -->
        <TextBlock DockPanel.Dock="Top"
                   Text="Редактирование книги"
                   FontSize="20"
                   FontWeight="SemiBold"
                   Margin="0,0,0,24" />

        <!-- Кнопки внизу — объявляем до контента, чтобы DockPanel их не
вытеснил -->
        <Border DockPanel.Dock="Bottom"
                BorderBrush="#E0E0E0"
                BorderThickness="0,1,0,0"
                Padding="0,16,0,0">
            <StackPanel Orientation="Horizontal"
                        HorizontalAlignment="Right">
                <Button Content="Отмена"
                        Width="90"
                        IsCancel="True" />
                <Button Content="Сохранить"
                        Width="90"
                        Margin="8,0,0,0"
                        IsDefault="True" />
            </StackPanel>
        </Border>

        <!-- Форма в центре, с прокруткой -->
        <ScrollViewer VerticalScrollBarVisibility="Auto">
            <StackPanel>

                <!-- Секция: основные данные -->
                <TextBlock Text="ОСНОВНЫЕ ДАННЫЕ"
```

```
                FontSize="11"
                FontWeight="SemiBold"
                Foreground="Gray"
                Margin="0,0,0,12" />

        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="90" />
                <ColumnDefinition Width="*" />
            </Grid.ColumnDefinitions>

            <TextBlock Text="Название:"
                       Grid.Row="0" Grid.Column="0"
                       VerticalAlignment="Center" />
            <TextBox   Grid.Row="0" Grid.Column="1"
                       MaxLength="200" />

            <TextBlock Text="Автор:"
                       Grid.Row="1" Grid.Column="0"
                       VerticalAlignment="Center"
                       Margin="0,8,0,0" />
            <TextBox   Grid.Row="1" Grid.Column="1"
                       Margin="0,8,0,0"
                       MaxLength="100" />

            <TextBlock Text="Год:"
                       Grid.Row="2" Grid.Column="0"
                       VerticalAlignment="Center"
                       Margin="0,8,0,0" />
            <TextBox   Grid.Row="2" Grid.Column="1"
                       Margin="0,8,0,0"
                       MaxLength="4"
                       Width="80"
                       HorizontalAlignment="Left" />

            <TextBlock Text="Жанр:"
                       Grid.Row="3" Grid.Column="0"
                       VerticalAlignment="Center"
                       Margin="0,8,0,0" />
            <ComboBox  Grid.Row="3" Grid.Column="1"
                       Margin="0,8,0,0"
                       HorizontalAlignment="Stretch">
                <ComboBoxItem Content="Классика" IsSelected="True"
/>
```

```
                <ComboBoxItem Content="Фантастика" />
                <ComboBoxItem Content="Детектив" />
                <ComboBoxItem Content="История" />
                <ComboBoxItem Content="Биография" />
            </ComboBox>
        </Grid>

        <Separator Margin="0,20,0,16" />

        <!-- Секция: описание -->
        <TextBlock Text="ОПИСАНИЕ"
                   FontSize="11"
                   FontWeight="SemiBold"
                   Foreground="Gray"
                   Margin="0,0,0,12" />

        <TextBox AcceptsReturn="True"
                 TextWrapping="Wrap"
                 Height="100"
                 VerticalScrollBarVisibility="Auto" />

        <Separator Margin="0,20,0,16" />

        <!-- Секция: настройки -->
        <TextBlock Text="НАСТРОЙКИ"
                   FontSize="11"
                   FontWeight="SemiBold"
                   Foreground="Gray"
                   Margin="0,0,0,12" />

        <StackPanel>
            <CheckBox Content="В наличии" />
            <CheckBox Content="Рекомендуемая книга"
                      Margin="0,8,0,0" />
            <CheckBox Content="Доступна для выдачи"
                      IsChecked="True"
                      Margin="0,8,0,0" />
        </StackPanel>

        <!-- ProgressBar скрыт — появится при сохранении (лекция 9)
-->
        <ProgressBar IsIndeterminate="True"
                     Height="4"
                     Margin="0,20,0,0"
                     Visibility="Collapsed" />

    </StackPanel>
  </ScrollViewer>

</DockPanel>
```

```
</Window>
```



Форма использует всё, что мы разобрали: `DockPanel` для общего layout, `Grid` для выравнивания меток и полей, `StackPanel` для группировки, `ScrollViewer` для прокрутки, `Separator` для визуального разделения секций и все основные элементы ввода.

Кнопки «Сохранить» и «Отмена» пока ничего не делают, `ProgressBar` скрыт через `Visibility="Collapsed"`. В лекции 9 мы подключим привязки и команды — форма оживёт: поля будут отображать реальные данные, кнопка «Сохранить» станет активной только когда заполнены обязательные поля, а `ProgressBar` будет появляться во время сохранения.

6. Стили и ресурсы

До сих пор мы задавали свойства элементов напрямую — каждому элементу по отдельности. Пока форма небольшая, это работает. Но представим, что в приложении много кнопок и нужно изменить шрифт у всех. Или все заголовки секций во всех частях окна должны выглядеть одинаково. Менять каждый элемент вручную — долго и ненадёжно.

WPF решает эту проблему через систему стилей и ресурсов.

Ресурсы

В WPF любой объект — стиль, кисть, число, строка — можно объявить как **ресурс** и затем использовать по ключу из любого места.

Ресурсы объявляются в секции `Resources` любого элемента:

```
<Window.Resources>
    <!-- Здесь объявляем ресурсы, доступные внутри этого окна -->
</Window.Resources>
```

Ресурсы наследуются по дереву элементов: ресурс на `Window` доступен всем элементам внутри окна. Ресурс на `StackPanel` — только элементам внутри этой панели. Для ресурсов, нужных во всём приложении, используется `App.xaml`:

```
<!-- App.xaml -->
<Application.Resources>
    <!-- Доступно в любом окне приложения -->
</Application.Resources>
```

StaticResource и DynamicResource

Ссылка на ресурс записывается двумя способами.

`StaticResource` — ресурс разрешается один раз при загрузке XAML. Быстрее, подходит для большинства случаев:

```
<Button Style="{StaticResource PrimaryButton}" />
```

`DynamicResource` — ресурс разрешается в рантайме и обновляется если ресурс изменился. Нужен когда ресурс может меняться во время работы приложения — например при смене темы:

```
<TextBlock Foreground="{DynamicResource AccentBrush}" />
```

В большинстве случаев используйте `StaticResource` — он проще и производительнее. `DynamicResource` нужен только когда ресурс действительно меняется в рантайме.

Простые ресурсы

Ресурсом может быть не только стиль, но и любое значение — цвет, кисть, число:

```
<Window.Resources>
    <SolidColorBrush x:Key="AccentBrush" Color="#2563EB" />
    <SolidColorBrush x:Key="DangerBrush" Color="#DC2626" />
    <SolidColorBrush x:Key="MutedBrush"  Color="#6B7280" />
</Window.Resources>
```

Использование:

```
<TextBlock Foreground="{StaticResource AccentBrush}"
           Text="Каталог книг" />

<Button Background="{StaticResource DangerBrush}"
        Content="Удалить" />
```

Теперь чтобы изменить акцентный цвет во всём окне — достаточно поправить одно значение в `Resources`.

Style — переиспользуемое оформление

`Style` объединяет набор значений свойств под одним ключом. Объявляется в `Resources`, применяется через атрибут `Style`:

```
<Window.Resources>

    <Style x:Key="SectionHeader" TargetType="TextBlock">
        <Setter Property="FontSize"   Value="11" />
        <Setter Property="FontWeight" Value="SemiBold" />
        <Setter Property="Foreground" Value="{StaticResource MutedBrush}" />
        <Setter Property="Margin"     Value="0,0,0,12" />
    </Style>

</Window.Resources>
```

```
<!-- Вместо четырёх атрибутов на каждом элементе — один -->
<TextBlock Text="ОСНОВНЫЕ ДАННЫЕ"
           Style="{StaticResource SectionHeader}" />
```

```
<TextBlock Text="ОПИСАНИЕ"
           Style="{StaticResource SectionHeader}" />
```

`TargetType` указывает, к какому типу элементов применяется стиль. Если попытаться применить стиль к элементу другого типа — получим ошибку. Это полезно: `TargetType` позволяет в `Setter` указывать свойства без квалификации типа.

Неявные стили

Стиль с `x:Key` применяется явно — нужно написать `Style="{StaticResource ...}"` на каждом элементе. Стиль **без** `x:Key` применяется автоматически ко всем элементам указанного типа в области видимости:

```
<Window.Resources>

    <!-- Применится ко всем TextBox в этом окне автоматически -->
    <Style TargetType="TextBox">
        <Setter Property="Height"  Value="32" />
        <Setter Property="Padding" Value="6,0" />
        <Setter Property="VerticalContentAlignment" Value="Center" />
    </Style>

</Window.Resources>
```

Теперь все `TextBox` в окне автоматически получат эти свойства без явного указания стиля. Неявные стили в `App.xaml` применяются ко всему приложению.

Наследование стилей

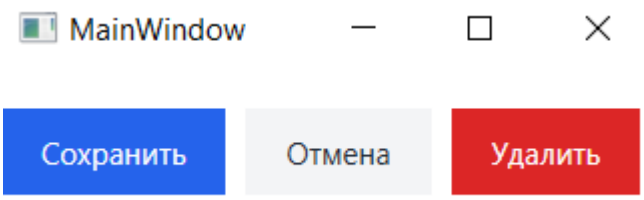
Стили можно строить друг на друге через `BasedOn`. Базовый стиль задаёт общие свойства, производные добавляют специфические:

```
<Window.Resources>

    <Style x:Key="BaseButton" TargetType="Button">
        <Setter Property="Height"          Value="34" />
        <Setter Property="Padding"         Value="16,0" />
        <Setter Property="Cursor"          Value="Hand" />
        <Setter Property="BorderThickness" Value="0" />
    </Style>

    <Style x:Key="PrimaryButton"
           TargetType="Button"
           BasedOn="{StaticResource BaseButton}">
        <Setter Property="Background" Value="#2563EB" />
        <Setter Property="Foreground" Value="White" />
    </Style>
```

```
    <Style x:Key="DangerButton"
           TargetType="Button"
           BasedOn="{StaticResource BaseButton}">
        <Setter Property="Background" Value="#DC2626" />
        <Setter Property="Foreground" Value="White" />
    </Style>

    <Style x:Key="SecondaryButton"
           TargetType="Button"
           BasedOn="{StaticResource BaseButton}">
        <Setter Property="Background" Value="#F3F4F6" />
        <Setter Property="Foreground" Value="#1F2937" />
    </Style>

</Window.Resources>
```

```
<StackPanel Orientation="Horizontal">
    <Button Content="Сохранить" Style="{StaticResource PrimaryButton}" />
    <Button Content="Отмена"    Style="{StaticResource SecondaryButton}"
            Margin="8,0,0,0" />
    <Button Content="Удалить"   Style="{StaticResource DangerButton}"
            Margin="8,0,0,0" />
</StackPanel>
```

Изменить высоту или отступы у всех кнопок сразу — достаточно поправить `BaseButton`.



Триггеры

Триггеры позволяют изменять свойства элемента в ответ на условие — без кода в code-behind. Объявляются внутри стиля в секции `Style.Triggers`.

`Trigger` — реагирует на значение свойства самого элемента:

```
<Style x:Key="PrimaryButton"
       TargetType="Button"
       BasedOn="{StaticResource BaseButton}">
    <Setter Property="Background" Value="#2563EB" />
    <Setter Property="Foreground" Value="White" />
    <Style.Triggers>
```

```
        <Trigger Property="IsMouseOver" Value="True">
            <Setter Property="Background" Value="#1D4ED8" />
        </Trigger>

        <Trigger Property="IsPressed" Value="True">
            <Setter Property="Background" Value="#1E40AF" />
        </Trigger>

        <Trigger Property="IsEnabled" Value="False">
            <Setter Property="Opacity" Value="0.5" />
        </Trigger>

    </Style.Triggers>
</Style>
```

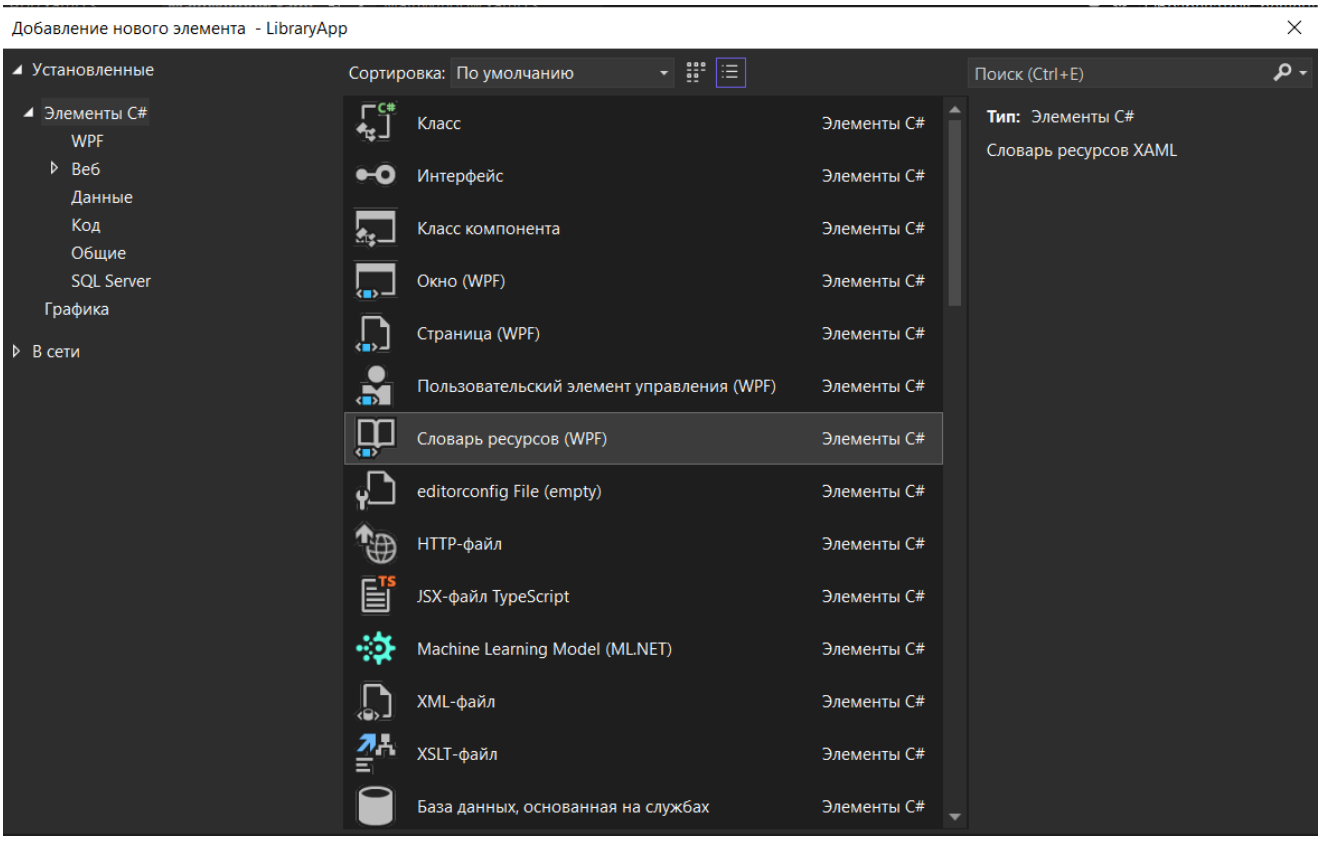
При наведении кнопка темнеет, при нажатии — ещё темнее, при отключении — полупрозрачная. Всё декларативно, без обработчиков событий.

`MultiTrigger` — срабатывает только когда выполняются несколько условий одновременно:

```
<Style.Triggers>
    <MultiTrigger>
        <MultiTrigger.Conditions>
            <Condition Property="IsMouseOver" Value="True" />
            <Condition Property="IsEnabled"   Value="True" />
        </MultiTrigger.Conditions>
        <Setter Property="Background" Value="#1D4ED8" />
    </MultiTrigger>
</Style.Triggers>
```

ResourceDictionary — стили в отдельном файле

Когда стилей становится много, держать их все в `Window.Resources` или `App.xaml` неудобно. WPF позволяет выносить ресурсы в отдельные файлы через `ResourceDictionary`.



Назовём его `ButtonStyles.xaml`:

```
<!-- Resources/ButtonStyles.xaml -->
<ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">

    <Style x:Key="BaseButton" TargetType="Button">
        <Setter Property="Height"          Value="34" />
        <Setter Property="Padding"         Value="16,0" />
        <Setter Property="Cursor"          Value="Hand" />
        <Setter Property="BorderThickness" Value="0" />
    </Style>

    <Style x:Key="PrimaryButton"
           TargetType="Button"
           BasedOn="{StaticResource BaseButton}">
        <Setter Property="Background" Value="#2563EB" />
        <Setter Property="Foreground" Value="White" />
        <!-- Триггеры для состояний -->
        <Style.Triggers>
            <Trigger Property="IsMouseOver" Value="True">
                <Setter Property="Background" Value="#1D4ED8" />
            </Trigger>
            <Trigger Property="IsPressed" Value="True">
                <Setter Property="Background" Value="#1E40AF" />
            </Trigger>
            <Trigger Property="IsEnabled" Value="False">
                <Setter Property="Opacity" Value="0.5" />
```

```
            </Trigger>
        </Style.Triggers>
    </Style>

    <!-- SecondaryButton, DangerButton — аналогично -->

</ResourceDictionary>
```

Подключаем словарь в `App.xaml` через `MergedDictionaries`:

```
<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <ResourceDictionary Source="Resources/ButtonStyles.xaml" />
            <ResourceDictionary Source="Resources/Colors.xaml" />
        </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
</Application.Resources>
```

После этого все стили из подключённых словарей доступны в любом окне приложения.

Применяем стили к форме

Вернёмся к форме из раздела 5 и применим стили. Добавим `Window.Resources` с неявным стилем для `TextBox` и именованным стилем для заголовков секций:

```
<Window.Resources>
    <SolidColorBrush x:Key="MutedBrush"  Color="#6B7280" />
    <SolidColorBrush x:Key="BorderBrush" Color="#E5E7EB" />

    <!-- Заголовки секций -->
    <Style x:Key="SectionHeader" TargetType="TextBlock">
        <Setter Property="FontSize"   Value="11" />
        <Setter Property="FontWeight" Value="SemiBold" />
        <Setter Property="Foreground" Value="{StaticResource MutedBrush}" />
        <Setter Property="Margin"     Value="0,0,0,12" />
    </Style>

    <!-- Неявный стиль для всех TextBox -->
    <Style TargetType="TextBox">
        <Setter Property="Height"                   Value="32" />
        <Setter Property="Padding"                  Value="8,0" />
        <Setter Property="VerticalContentAlignment" Value="Center" />
        <Setter Property="BorderBrush"              Value="{StaticResource
BorderBrush}" />
    </Style>
</Window.Resources>
```

После применения заголовки секций упростились:

```
<!-- Было: четыре атрибута на каждом заголовке -->
<TextBlock Text="ОСНОВНЫЕ ДАННЫЕ"
           FontSize="11" FontWeight="SemiBold"
           Foreground="Gray" Margin="0,0,0,12" />

<!-- Стало: один атрибут -->
<TextBlock Text="ОСНОВНЫЕ ДАННЫЕ"
           Style="{StaticResource SectionHeader}" />
```

Кнопки формы теперь используют стили из `ButtonStyles.xaml`:

```
<!-- Было -->
<Button Content="Отмена"    Width="90" IsCancel="True" />
<Button Content="Сохранить" Width="90" Margin="8,0,0,0" IsDefault="True" />

<!-- Стало -->
<Button Content="Отмена"
        Width="90"
        Style="{StaticResource SecondaryButton}"
        IsCancel="True" />
<Button Content="Сохранить"
        Width="90"
        Style="{StaticResource PrimaryButton}"
        Margin="8,0,0,0"
        IsDefault="True" />
```

Все `TextBox` формы получили единую высоту и отступы автоматически — через неявный стиль, без изменений в разметке полей. Чтобы изменить цвет акцента или высоту всех полей, достаточно поправить одно место в ресурсах.